Fake News Detection

Group: Group 2B

1. Introduction

1.1 Business Understanding

In today's information-driven society, the spread of misinformation and fake news through online news dissemination platforms and social media platforms has become a serious threat to public trust, democracy, health communication, and media integrity. The ability to automatically distinguish fake from real news is essential for:

- Social media platforms (e.g., Meta, Twitter/X, Reddit).
- Fact-checking organizations.
- News aggregators (e.g., Google News, Apple News).
- The general public.

1.2 Problem Statement:

The proliferation of fake news on digital platforms poses a significant risk to public trust and decision-making. Traditional methods of identifying fake news are slow and resource-intensive, making it difficult to manage the vast volume of online content.

1.3 Proposed solution

This project aims to develop an automated binary classification system using transformer-based Natural Language Processing (NLP) models to detect and flag fake news articles with high accuracy. By leveraging advanced NLP techniques, the system will offer a scalable solution to help identify harmful misinformation, improving information credibility and supporting the fight against digital manipulation.

1.4 Objectives

1. Build a Baseline Model to predict class of an article.
2. Build an Ensemble Model to predict class of an article.
3. Build an LSTM Neural Network to predict class of an article.
4. Leverage Transfer Learning to Finetune the RoBERTa pretrained transformer.
5. Select the best-fit, most-robust, and generalizable model for deployment.

1.5 Success Criteria

Because the classes for the target variable of the dataset used in this project is imbalanced; out success criteria is the ***Macro-averaged F1-score***. The performance metric is ideal for imbalanced datasets because it calculates the F1-score for each class independently and takes the unweighted mean, giving equal importance to all classes regardless of size. Additionally, the metric provides a balanced measure of precision and recall across all classes, enabling fair model evaluation and comparison when class distribution is skewed. We aim to achieve a ***Macro-averaged F1-score*** of over **76%** for the deployed model.

2. Data Loading, Feature Engineering, and Preprocessing

2.1 Data Loading

In [2]:

```
# Import basic libraries
import pandas as pd
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt
from wordcloud import WordCloud
from collections import Counter
import io

# Import NLP tools and NLTK library modules
import nltk
from nltk.corpus import stopwords, wordnet
from nltk.tokenize import word_tokenize, sent_tokenize
from nltk.stem import WordNetLemmatizer, PorterStemmer
from nltk import pos_tag
from nltk.util import bigrams
from nltk.util import trigrams
import string
import re
import spacy

# Import scikit-learn library's classes, tools, and modules
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler, LabelEncoder
from sklearn.model_selection import train_test_split, cross_val_score
from sklearn.feature_extraction.text import CountVectorizer, TfidfVectorize
from sklearn.metrics import accuracy_score, classification_report, confusion
from sklearn.metrics import f1_score, precision_score, recall_score, auc, ro
from sklearn.linear_model import LogisticRegression
from xgboost import XGBClassifier
from sklearn.utils.class_weight import compute_class_weight
import joblib
from gensim.models import Word2Vec

# Import tensorflow, and keras modules
import tensorflow as tf
from tensorflow import keras
from tensorflow.keras.preprocessing.text import Tokenizer
from tensorflow.keras.preprocessing.sequence import pad_sequences
from tensorflow.keras.models import Sequential
from tensorflow.keras.utils import to_categorical
from tensorflow.keras.models import Model, load_model
from tensorflow.keras.layers import TextVectorization, Embedding, GlobalAve
from tensorflow.keras.layers import Dropout, Dense, LSTM, Input, Masking
from tensorflow.keras.metrics import Precision, Recall, AUC
from transformers import TFRobertaForSequenceClassification, RobertaTokenize
from transformers import logging as transformers_logging

# Import transformers and pytorch modules
from transformers import RobertaTokenizerFast, RobertaForSequenceClassificat
from datasets import Dataset
import torch
from torch.utils.data import Dataset
import os

# Disable warnings
import warnings
warnings.filterwarnings('ignore')
```

In [2]:
```
data = pd.read_csv("/home/trigger/Documents/Flatiron/phase5/project/Fake-New
data.head()
```

Out[2]:

	news_url	title	extracted_article_text	news_type	class
0	https://www.bustle.com/p/will-the-royals-retur...	Will 'The Royals' Return For Season 5? This St...	Entertainment With a royal wedding and a coup ...	gossip	1
1	https://www.foxnews.com/entertainment/naya-riv...	Naya Rivera refiles for divorce from Ryan Dors...	This material may not be published, broadcast,...	gossip	1
2	https://www.unitedbypop.com/style/fashion/tayl...	Outfit ideas for Taylor Swift's Reputation Tour	United By Pop - United Kingdom. United States....	gossip	1
3	https://variety.com/2018/music/news/scott-hutc...	Scott Hutchison Dead: Frightened Rabbit Frontm...	By Robert Mitchell A body found Thursday night...	gossip	1
4	https://www.etonline.com/kate-middleton-gives-...	Kate Middleton Gives Birth to Royal Baby No. 3...	Baby No. 3 is officially here! Kate Middleton ...	gossip	1

In [3]:
```
# Check shape
data.shape
```

Out[3]:
```
(16044, 5)
```

In [4]:
```
# Check duplicates
data.duplicated().sum()
```

Out[4]:
```
928
```

In [5]:
```
# Drop duplicates
data.drop_duplicates(inplace=True)
```

In [6]:
```
# Confirm no duplicates
data.duplicated().sum()
```

Out[6]:
```
0
```

In [7]:
```
# Inspect column attributes to check missing values
data.info()
```

```
<class 'pandas.core.frame.DataFrame'>
Index: 15116 entries, 0 to 16043
Data columns (total 5 columns):
 #   Column                  Non-Null Count  Dtype 
---  ------                  --------------  ----- 
 0   news_url                15116 non-null  object
 1   title                   15116 non-null  object
 2   extracted_article_text  15116 non-null  object
 3   news_type               15116 non-null  object
 4   class                   15116 non-null  int64 
dtypes: int64(1), object(4)
memory usage: 708.6+ KB
```

- Data was successfully loaded and duplicates removed.
- There are no entries with missing values

2.2 Feature Engineering

In [9]:
```
# Engineer the domain feature
from urllib.parse import urlparse
data['domain'] = data['news_url'].apply(lambda x: urlparse(x).netloc if isi

# Engineer the article feature
data['article'] = data['title'] + " " + data['extracted_article_text']

# Engineer the text length
data['num_sentences']=data['extracted_article_text'].apply(lambda x: len(se

# Engineer the text length
data['text_length']=data['extracted_article_text'].apply(lambda x: len(x.sp
data.head(2)
```

Out[9]:

	news_url	title	extracted_article_text	news_type	class	
0	https://www.bustle.com/p/will-the-royals-retur...	Will 'The Royals' Return For Season 5? This St...	Entertainment With a royal wedding and a coup ...	gossip	1	www.bu
1	https://www.foxnews.com/entertainment/naya-riv...	Naya Rivera refiles for divorce from Ryan Dors...	This material may not be published, broadcast,...	gossip	1	www.foxn

In [10]:
```
data['domain'].value_counts().nlargest(5)
```

Out[10]:
```
domain
missing                3237
people.com             1224
www.dailymail.co.uk     682
www.etonline.com        536
www.usmagazine.com      512
Name: count, dtype: int64
```

2.3 Data Preprocessing

- Define a function for ***text cleaning, normalization, word tokenizing, Part-of-Speech tagging, lemmatization*** and ***removing stopwords***, using the **spaCy** library.

In [11]:
```
# Load spaCy model
nlp = spacy.load('en_core_web_sm', disable=['parser', 'ner'])
# Define function to clean text using spaCy
def cleaned_text(text, debug=False):
    # Convert lists to strings if necessary
    if isinstance(text, list):
        text = " ".join(text)
    # Ensure text is a string
    if not isinstance(text, str):
        text = str(text)
    text = text.lower()# Lowercase all characters
    text = re.sub(r'https?://\S+|www\.\S+', '', text) # Remove URLs
    text = re.sub(r'\[.*?\]', '', text) # Remove content enclosed by square
    text = re.sub(r'<.*?>+', '', text)  # Remove content enclosed with angl
    text = re.sub(r'\n', ' ', text)     # Replace newline characters with s

    # Define chunk size
    chunk_size = 900000
    cleaned_tokens = []
    # Process text in chunks if it's too long
    if len(text) > chunk_size:
        chunks = [text[i:i + chunk_size] for i in range(0, len(text), chunk_
    else:
        chunks = [text]

    for chunk in chunks:
        doc = nlp(chunk)
        # Extract lemmatized tokens, excluding stopwords,and punctuation
        for token in doc:
            if (not token.is_stop and
                not token.is_punct):
                #not token.is_digit and
                #len(token.lemma_) > 2):
                # Remove any remaining non-alphabetic characters from lemma
                cleaned = re.sub(r'[^a-zA-Z]', '', token.lemma_)
                if cleaned:  # Only include non-empty strings
                    cleaned_tokens.append(cleaned)
    return cleaned_tokens

# Apply the clean_text function to the 'spacy' column
data['spacy'] = data['article'].apply(cleaned_text)
```

In [12]:
```
# Join tokens
data['text'] = data['spacy'].apply(lambda x: ' '.join(x))
data.head(2)
```

Out[12]:

	news_url	title	extracted_article_text	news_type	class	
0	https://www.bustle.com/p/will-the-royals-retur...	Will 'The Royals' Return For Season 5? This St...	Entertainment With a royal wedding and a coup ...	gossip	1	www.bu
1	https://www.foxnews.com/entertainment/naya-riv...	Naya Rivera refiles for divorce from Ryan Dors...	This material may not be published, broadcast,...	gossip	1	www.foxr

In [13]:
```
# Preview a random sample
data.iloc[99, 10]
```

Out[13]:
```
'blac chyna cozy hot felon jeremy meek pic look like blac chyna hot felon
jeremy meek work chyna share picture posing arm snapchat wednesday night s
how tattoo chyna don skintight lace orange dress meek keep casual camoufla
ge print shirt rip jean news early snapchat post chyna appear photo shoot
lead speculate possibly star campaign meek model attractive mugshot go vir
al headline summer relationship topshop heiress chloe green meek photograp
h kiss green luxury yacht coast turkey july married wife melissa time shor
tly picture surface meek file separation melissa year marriage news meek g
reen shy pack pda watch video'
```

In [14]:
```
data.info()
```

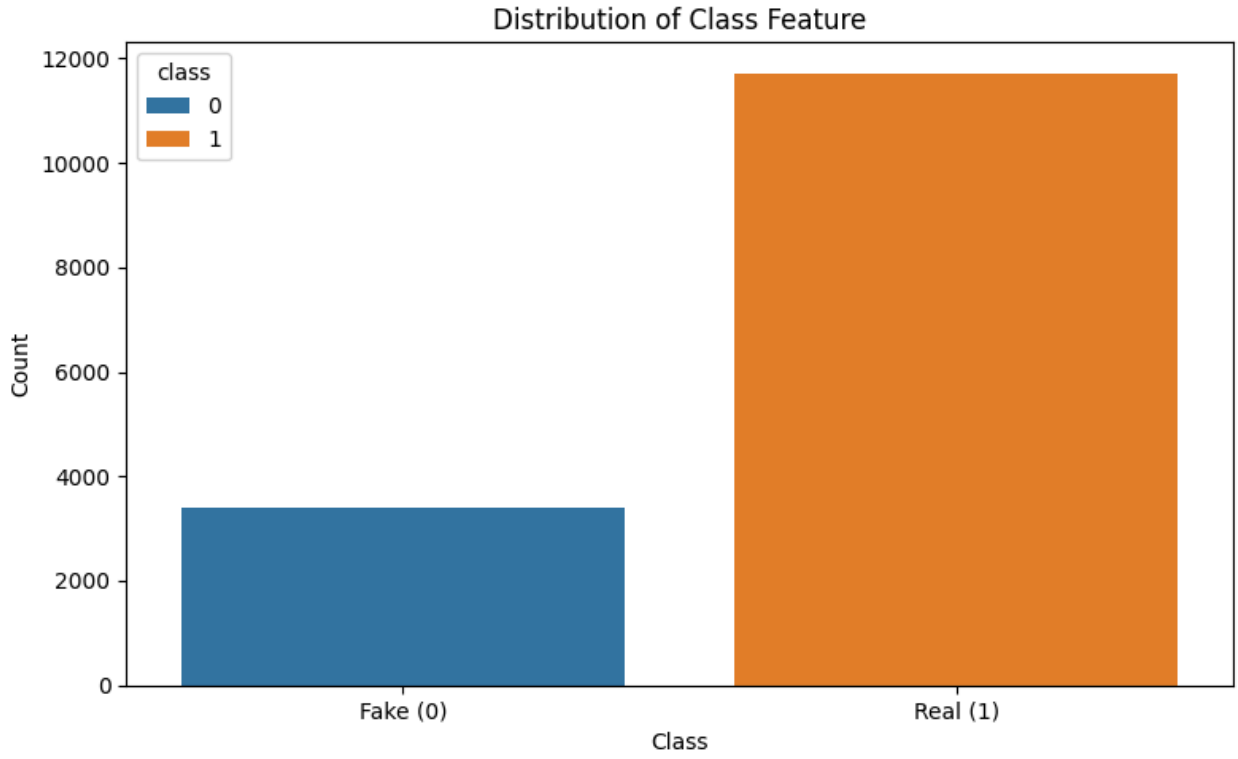
```
<class 'pandas.core.frame.DataFrame'>
Index: 15116 entries, 0 to 16043
Data columns (total 11 columns):
 #   Column                  Non-Null Count  Dtype
---  ------                  --------------  -----
 0   news_url                15116 non-null  object
 1   title                   15116 non-null  object
 2   extracted_article_text  15116 non-null  object
 3   news_type               15116 non-null  object
 4   class                   15116 non-null  int64
 5   domain                  15116 non-null  object
 6   article                 15116 non-null  object
 7   num_sentences           15116 non-null  int64
 8   text_length             15116 non-null  int64
 9   spacy                   15116 non-null  object
 10  text                    15116 non-null  object
dtypes: int64(3), object(8)
memory usage: 1.4+ MB
```

2.4 Extraploratory Data Analysis

2.4.1 Distributions

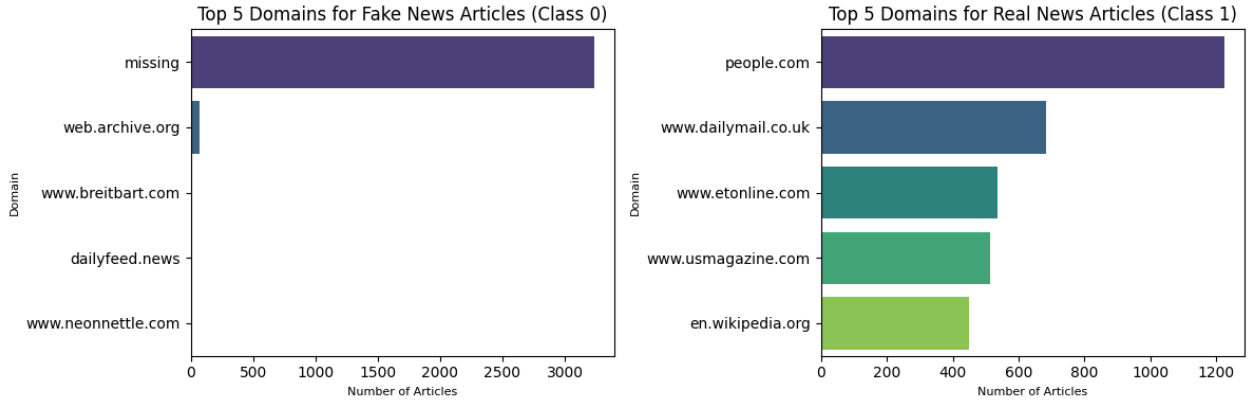
In [33]:

```
# Visualize Class Distribuction
plt.figure(figsize=(8, 5))
sns.barplot(x='class', y='class', data=data, estimator=len, hue='class')

# Add labels and title
plt.xlabel('Class')
plt.ylabel('Count')
plt.title('Distribution of Class Feature')
plt.xticks([0, 1], ['Fake (0)', 'Real (1)'])
plt.tight_layout()
plt.savefig('./images/class-distributions.png', dpi=300, bbox_inches='tight
plt.show()
```



In [34]:

```
# Visualize top 5 domains for fake news and real news
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(12, 4))

# Plot for class 0 (Fake News)
fake_data = data[data['class'] == 0]
domain_counts = fake_data['domain'].value_counts()
top_5_domains = domain_counts.nlargest(5)
sns.barplot(x=top_5_domains.values, y=top_5_domains.index, palette='viridis
ax1.set_title('Top 5 Domains for Fake News Articles (Class 0)', fontsize=12
ax1.set_xlabel('Number of Articles', fontsize=8)
ax1.set_ylabel('Domain', fontsize=8)

# Plot for class 1 (Real News)
fake_data = data[data['class'] == 1]
domain_counts = fake_data['domain'].value_counts()
top_5_domains = domain_counts.nlargest(5)
sns.barplot(x=top_5_domains.values, y=top_5_domains.index, palette='viridis
ax2.set_title('Top 5 Domains for Real News Articles (Class 1)', fontsize=12
ax2.set_xlabel('Number of Articles', fontsize=8)
ax2.set_ylabel('Domain', fontsize=8)
plt.tight_layout()
plt.savefig('./images/top-5-domains-for-fake-and-real-news.png', dpi=300, bb
plt.show()
```



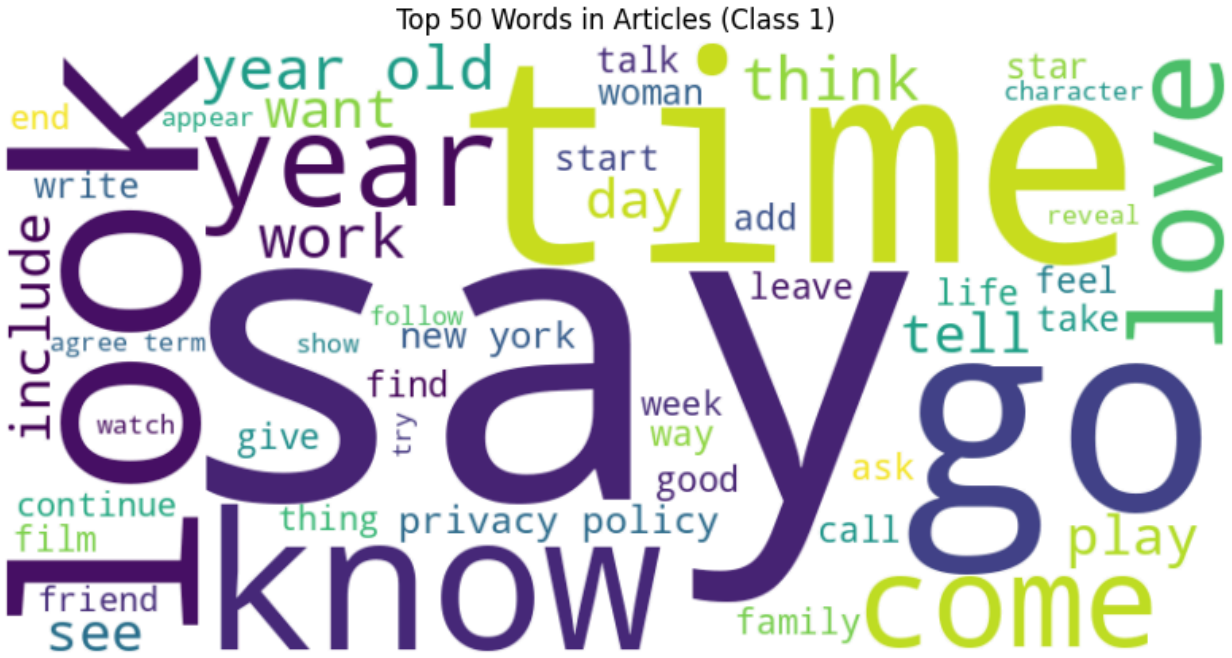
2.4.2 Word Clouds

In [35]:

```
# Filter data for class 0
class_0_data = data[data['class'] == 0]

# Combine all text from extracted_article_text
text = ' '.join(class_0_data['text'].dropna().astype(str))
wordcloud = WordCloud(width=800, height=400, background_color='white', max_w
plt.figure(figsize=(10, 5))
plt.imshow(wordcloud, interpolation='bilinear')
plt.axis('off')
plt.title('Top 50 Words in Articles (Class 0)')
plt.savefig('./images/top-50-words-for-class0-wordcloud-plot.png', dpi=300,
plt.show()
```

In [36]:

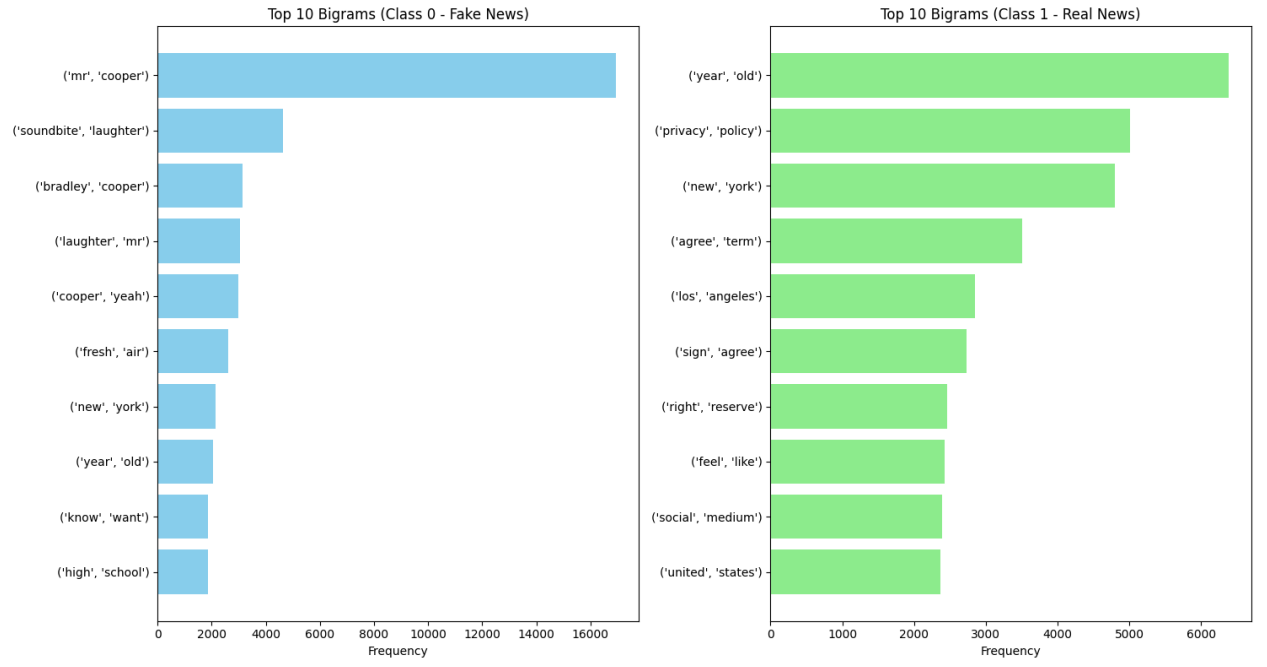
```
# Filter data for class 1
class_0_data = data[data['class'] == 1]

# Combine all text from extracted_article_text
text = ' '.join(class_0_data['text'].dropna().astype(str))
wordcloud = WordCloud(width=800, height=400, background_color='white', max_w
plt.figure(figsize=(10, 5))
plt.imshow(wordcloud, interpolation='bilinear')
plt.axis('off')
plt.title('Top 50 Words in Articles (Class 1)')
plt.savefig('./images/top-50-words-for-class1-wordcloud-plot.png', dpi=300,
plt.show()
```



2.4.3 Top N-grams

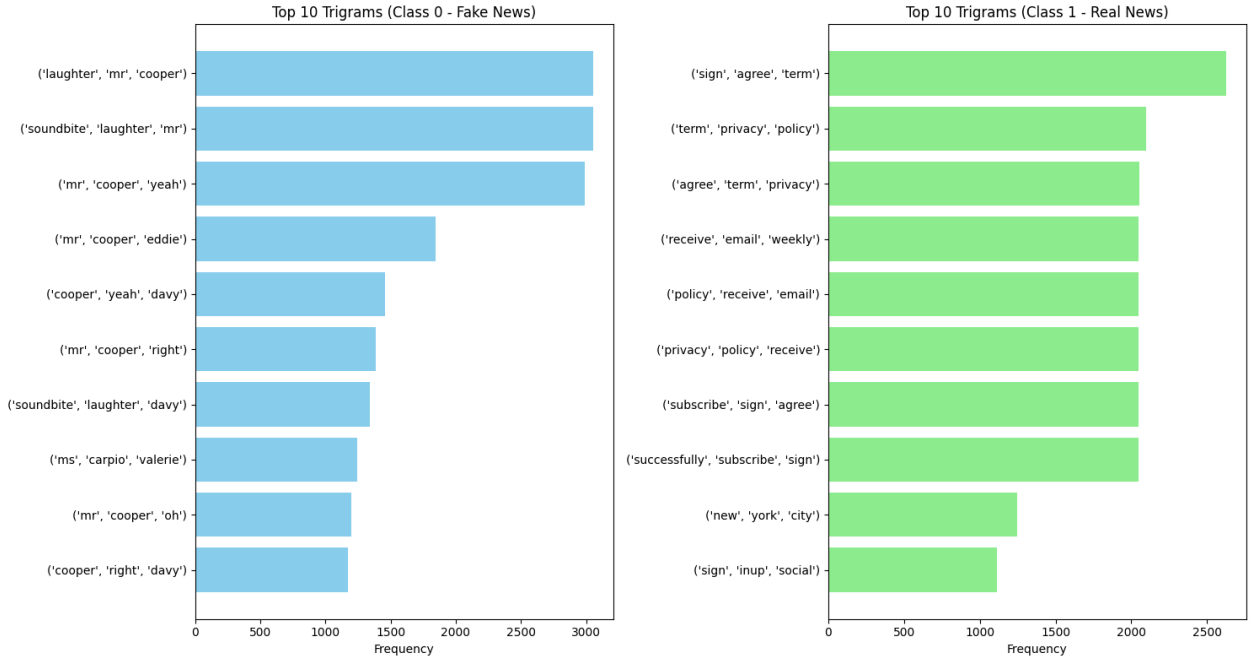
In [37]:

```
# Filter and plot top-10 bigrams for both fake and real news articles
def get_top_bigrams(text_series):
    all_bigrams = []
    for text in text_series.dropna():
        tokens = word_tokenize(text.lower())
        bigram_list = list(bigrams(tokens))
        all_bigrams.extend(bigram_list)
    return Counter(all_bigrams).most_common(10)

class_0_data = data[data['class'] == 0]['text']
class_1_data = data[data['class'] == 1]['text']

# Get top 10 bigrams for each class
top_bigrams_class_0 = get_top_bigrams(class_0_data)
top_bigrams_class_1 = get_top_bigrams(class_1_data)
bigrams_0, counts_0 = zip(*sorted(top_bigrams_class_0, key=lambda x: x[1], 
bigrams_1, counts_1 = zip(*sorted(top_bigrams_class_1, key=lambda x: x[1], 

# Create subplots
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(15, 8))
# Plot for class 0
ax1.barh([str(bg) for bg in bigrams_0], counts_0, color='skyblue')
ax1.set_title('Top 10 Bigrams (Class 0 - Fake News)')
ax1.set_xlabel('Frequency')
# Plot for class 1
ax2.barh([str(bg) for bg in bigrams_1], counts_1, color='lightgreen')
ax2.set_title('Top 10 Bigrams (Class 1 - Real News)')
ax2.set_xlabel('Frequency')
plt.tight_layout()
plt.savefig('./images/top-10-bigrams.png', dpi=300, bbox_inches='tight')
plt.show()
```



In [38]:

```
# Filter and plot top-10 trigrams for both fake and real news articles
def get_top_trigrams(text_series):
    all_trigrams = []
    for text in text_series.dropna():
        tokens = word_tokenize(text.lower())
        trigram_list = list(trigrams(tokens))
        all_trigrams.extend(trigram_list)
    return Counter(all_trigrams).most_common(10)

# Filter data for class 0 and class 1
class_0_data = data[data['class'] == 0]['text']
class_1_data = data[data['class'] == 1]['text']
top_trigrams_class_0 = get_top_trigrams(class_0_data)
top_trigrams_class_1 = get_top_trigrams(class_1_data)
trigrams_0, counts_0 = zip(*sorted(top_trigrams_class_0, key=lambda x: x[1]
trigrams_1, counts_1 = zip(*sorted(top_trigrams_class_1, key=lambda x: x[1]

# Create subplots
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(15, 8))
# Plot for class 0
ax1.barh([str(tg) for tg in trigrams_0], counts_0, color='skyblue')
ax1.set_title('Top 10 Trigrams (Class 0 - Fake News)')
ax1.set_xlabel('Frequency')
# Plot for class 1
ax2.barh([str(tg) for tg in trigrams_1], counts_1, color='lightgreen')
ax2.set_title('Top 10 Trigrams (Class 1 - Real News)')
ax2.set_xlabel('Frequency')
plt.tight_layout()
plt.savefig('./images/top-10-trigrams.png', dpi=300, bbox_inches='tight')
plt.show()
```



3. Modelling

In [29]:
```
# Create a copy of data
df = data.copy()

# Define endog and exog
endog = df["class"]
exog = df[['text']]

# First train-test split (70-30)
X_temp, X_test, y_temp, y_test = train_test_split(exog, endog, test_size=0.3

# Second split temp into train and validation (70-30 of remaining 70%)
X_train, X_val, y_train, y_val = train_test_split(X_temp, y_temp, test_size=
```

In [30]:
```
# Tokenize text feature for Word2Vec
def tokenize_text(text_series):
    return [word_tokenize(text.lower()) for text in text_series.dropna()]
tokenized_spacy_text = tokenize_text(exog['text'])

# Train Word2Vec model on the tokenized text
word2vec_model = Word2Vec(sentences=tokenized_spacy_text, vector_size=100, w

# Function to get average Word2Vec vector for a text
def get_word2vec_vector(text, model, vector_size=100):
    words = word_tokenize(text.lower())
    word_vectors = [model.wv[word] for word in words if word in model.wv]
    if len(word_vectors) == 0:
        return np.zeros(vector_size)
    return np.mean(word_vectors, axis=0)

# Vectorize spacy_text using Word2Vec
X_train_1 = np.array([get_word2vec_vector(text, word2vec_model) for text in
X_val_1 = np.array([get_word2vec_vector(text, word2vec_model) for text in X_
X_test_1 = np.array([get_word2vec_vector(text, word2vec_model) for text in X

# Check shapes
print("X_train shape:", X_train_1.shape)
print("X_val shape:", X_val_1.shape)
print("X_test shape:", X_test_1.shape)
```

```
X_train shape: (7406, 100)
X_val shape: (3175, 100)
X_test shape: (4535, 100)
```

Objective 1: Build a Baseline Model to Predict Class of an Article

In [23]:

```
# Initialize and fit a logistic regression model
X_train_logistic = np.vstack((X_train_1, X_val_1))
y_train_logistic = np.hstack((y_train, y_val))

# Initialize the logistic regression model
lr_model = LogisticRegression(max_iter=1000, random_state=42)

# Perform 5-fold cross-validation on the combined train + validation set
cv_scores = cross_val_score(lr_model, X_train_logistic, y_train_logistic, cv
print("Cross-Validation Scores:", cv_scores)
print("Average CV Score (F1):", cv_scores.mean())
print("Standard Deviation:", cv_scores.std())

# Fit the model and predict on test set
lr_model.fit(X_train_logistic, y_train_logistic)
```

```
Cross-Validation Scores: [0.90457369 0.90339128 0.89204545 0.90302516 0.89
769165]
Average CV Score (F1): 0.900145446780912
Standard Deviation: 0.004691875940038059
```

Out[23]:

```
LogisticRegression(max_iter=1000, random_state=42)
```

In a Jupyter environment, please rerun this cell to show the HTML representation or trust the notebook.
On GitHub, the HTML representation is unable to render, please try loading this page with nbviewer.org.

Objective 2: Build an Ensemble Model to Predict Class of an Article

In [24]:

```
# Initialize and fit an XGBoost model
X_train_xgb = np.vstack((X_train_1, X_val_1))
y_train_xgb = np.hstack((y_train, y_val))

# Initialize the XGBoost model
xgb_model = XGBClassifier(use_label_encoder=False, eval_metric='logloss', ra

# Perform 5-fold cross-validation on the combined train + validation set
cv_scores = cross_val_score(xgb_model, X_train_xgb, y_train_xgb, cv=5, scori
print("Cross-Validation Scores:", cv_scores)
print("Average CV Score (F1):", cv_scores.mean())
print("Standard Deviation:", cv_scores.std())

# Fit the model on the entire train + validation set
xgb_model.fit(X_train_xgb, y_train_xgb)
```

```
Cross-Validation Scores: [0.9039678  0.90515999 0.89824157 0.90080275 0.89
971182]
Average CV Score (F1): 0.9015767844169847
Standard Deviation: 0.002598353216426927
```

Out[24]:

```
XGBClassifier(base_score=None, booster=None, callbacks=None,
              colsample_bylevel=None, colsample_bynode=None,
              colsample_bytree=None, device=None, early_stopping_rounds=No
ne,
              enable_categorical=False, eval_metric='logloss',
              feature_types=None, gamma=None, grow_policy=None,
              importance_type=None, interaction_constraints=None,
              learning_rate=None, max_bin=None, max_cat_threshold=None,
              max_cat_to_onehot=None, max_delta_step=None, max_depth=None,
              max_leaves=None, min_child_weight=None, missing=nan,
              monotone_constraints=None, multi_strategy=None, n_estimators
=None,
              n_jobs=None, num_parallel_tree=None, random_state=42, ...)
```

In a Jupyter environment, please rerun this cell to show the HTML representation or trust the notebook.
On GitHub, the HTML representation is unable to render, please try loading this page with nbviewer.org.

Objective 3: Build an LSTM Neural Network to Predict Class of an Article

In [25]:
```
# Initialize and fit an LSTM neural network using Keras functional API

# Define Parameters
max_len = 300  # Max tokens per sample
embedding_dim = 100  # As set in Word2Vec

# Convert text to padded sequences of Word2Vec vectors
def text_to_vector_sequence(text, model, max_len=300, embedding_dim=100):
    tokens = word_tokenize(text.lower())
    sequence = [model.wv[token] for token in tokens if token in model.wv]

    if len(sequence) == 0:
        return np.zeros((max_len, embedding_dim))

    # Truncate if too long
    sequence = sequence[:max_len]

    # Pad if too short
    if len(sequence) < max_len:
        padding = np.zeros((max_len - len(sequence), embedding_dim))
        sequence = np.vstack((sequence, padding))

    return np.array(sequence)

# Vectorize all text sets into 3D arrays
X_train_seq = np.array([text_to_vector_sequence(text, word2vec_model, max_le
X_val_seq = np.array([text_to_vector_sequence(text, word2vec_model, max_len
X_test_seq = np.array([text_to_vector_sequence(text, word2vec_model, max_le
```

In [26]:
```
# Custom F1 Score Metric
class F1Score(tf.keras.metrics.Metric):
    def __init__(self, name='f1_score', **kwargs):
        super().__init__(name=name, **kwargs)
        self.precision = Precision()
        self.recall = Recall()

    def update_state(self, y_true, y_pred, sample_weight=None):
        y_pred_binary = tf.round(y_pred)  # Convert probabilities to binary
        self.precision.update_state(y_true, y_pred_binary, sample_weight)
        self.recall.update_state(y_true, y_pred_binary, sample_weight)

    def result(self):
        p = self.precision.result()
        r = self.recall.result()
        return 2 * ((p * r) / (p + r + tf.keras.backend.epsilon()))

    def reset_state(self):
        self.precision.reset_state()
        self.recall.reset_state()
```

In [27]:

```python
# Define LSTM architecture
inputs = Input(shape=(max_len, embedding_dim))
x = Masking(mask_value=0.0)(inputs)  # Handle zero-padding
x = LSTM(64)(x)
x = Dropout(0.3)(x)
x = Dense(32, activation='relu')(x)
outputs = Dense(1, activation='sigmoid')(x)

# Compile the model with metrics
lstm_model = Model(inputs, outputs)
lstm_model.compile(
    optimizer='adam',
    loss='binary_crossentropy',
    metrics=[
        # 'accuracy',                    # Built-in accuracy
        # Precision(name='precision'),   # Precision metric
        # Recall(name='recall'),         # Recall metric
        F1Score(name='f1_score'),      # Custom F1 score
        # AUC(name='roc_auc')            # ROC-AUC score
    ])
```

```
I0000 00:00:1754430837.690688      36 gpu_device.cc:2022] Created device /
job:localhost/replica:0/task:0/device:GPU:0 with 13942 MB memory:  -> devi
ce: 0, name: Tesla T4, pci bus id: 0000:00:04.0, compute capability: 7.5
I0000 00:00:1754430837.691379      36 gpu_device.cc:2022] Created device /
job:localhost/replica:0/task:0/device:GPU:1 with 13942 MB memory:  -> devi
ce: 1, name: Tesla T4, pci bus id: 0000:00:05.0, compute capability: 7.5
```

In [28]:

```
# Fit
history = lstm_model.fit(
    X_train_seq, y_train,
    validation_data=(X_val_seq, y_val),
    epochs=10,
    batch_size=64)
```

```
Epoch 1/10

I0000 00:00:1754430846.386353     134 cuda_dnn.cc:529] Loaded cuDNN versio
n 90300

116/116 ━━━━━━━━━━━━━━━━━━━━ 10s 36ms/step - f1_score: 0.8637 - loss: 0.53
98 - val_f1_score: 0.8852 - val_loss: 0.4726
Epoch 2/10
116/116 ━━━━━━━━━━━━━━━━━━━━ 2s 19ms/step - f1_score: 0.8822 - loss: 0.475
9 - val_f1_score: 0.8899 - val_loss: 0.4534
Epoch 3/10
116/116 ━━━━━━━━━━━━━━━━━━━━ 2s 19ms/step - f1_score: 0.8935 - loss: 0.431
6 - val_f1_score: 0.8880 - val_loss: 0.4210
Epoch 4/10
116/116 ━━━━━━━━━━━━━━━━━━━━ 2s 20ms/step - f1_score: 0.8973 - loss: 0.417
7 - val_f1_score: 0.8941 - val_loss: 0.4391
Epoch 5/10
116/116 ━━━━━━━━━━━━━━━━━━━━ 2s 19ms/step - f1_score: 0.8908 - loss: 0.431
1 - val_f1_score: 0.8936 - val_loss: 0.4284
Epoch 6/10
116/116 ━━━━━━━━━━━━━━━━━━━━ 2s 19ms/step - f1_score: 0.8965 - loss: 0.409
7 - val_f1_score: 0.8950 - val_loss: 0.4051
Epoch 7/10
116/116 ━━━━━━━━━━━━━━━━━━━━ 2s 19ms/step - f1_score: 0.9076 - loss: 0.363
7 - val_f1_score: 0.8975 - val_loss: 0.4164
Epoch 8/10
116/116 ━━━━━━━━━━━━━━━━━━━━ 2s 19ms/step - f1_score: 0.9146 - loss: 0.355
8 - val_f1_score: 0.8962 - val_loss: 0.4292
Epoch 9/10
116/116 ━━━━━━━━━━━━━━━━━━━━ 2s 18ms/step - f1_score: 0.9121 - loss: 0.339
7 - val_f1_score: 0.8956 - val_loss: 0.4277
Epoch 10/10
116/116 ━━━━━━━━━━━━━━━━━━━━ 2s 18ms/step - f1_score: 0.9143 - loss: 0.344
5 - val_f1_score: 0.8909 - val_loss: 0.4151
```

Objective 4: Leverage Transfer Learning to Finetune the RoBERTa Pretrained Transformer

RoBERTa

In [31]:

```
from transformers import RobertaTokenizer, RobertaForSequenceClassification

# Set device
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
print(f"Using device: {device}")

# Initialize RoBERTa model and tokenizer
model_name = "roberta-base"
roberta_tokenizer = RobertaTokenizer.from_pretrained(model_name)
roberta_model = RobertaForSequenceClassification.from_pretrained(
    model_name,
    num_labels=2,
    hidden_dropout_prob=0.2,
    attention_probs_dropout_prob=0.2
).to(device)
```

```
Using device: cpu

Some weights of RobertaForSequenceClassification were not initialized from
the model checkpoint at roberta-base and are newly initialized: ['classifi
er.dense.bias', 'classifier.dense.weight', 'classifier.out_proj.bias', 'cl
assifier.out_proj.weight']
You should probably TRAIN this model on a down-stream task to be able to u
se it for predictions and inference.
```

In [30]:

```
# Initialize the NewsDataset with text chunking using a sliding window
class NewsDataset(Dataset):
    def __init__(self, texts, labels, tokenizer, max_length=512, stride=256
        self.tokenizer = tokenizer
        self.max_length = max_length
        self.stride = stride

        self.all_chunk_encodings = []    # Store {'input_ids', 'attention_ma
        self.all_chunk_labels = []       # Store the label for each chunk
        self.chunk_original_indices = [] # Store the index of the original a

        # Process each original text to create chunks
        for original_idx, text in enumerate(texts):
            tokenized_output = tokenizer(
                text,
                truncation=False,
                padding=False,
                max_length=self.max_length,
                stride=self.stride,
                return_overflowing_tokens=True,
                return_tensors="pt" # Return PyTorch tensors for chunks
            )

            # Iterate through each chunk generated for the current original
            for i in range(len(tokenized_output['input_ids'])):
                chunk_input_ids = tokenized_output['input_ids'][i]
                chunk_attention_mask = tokenized_output['attention_mask'][i]
                if chunk_input_ids.shape[0] > self.max_length:
                    # Truncate if the chunk is longer than max_length
                    chunk_input_ids = chunk_input_ids[:self.max_length]
                    chunk_attention_mask = chunk_attention_mask[:self.max_le
                elif chunk_input_ids.shape[0] < self.max_length:
                    # Pad if the chunk is shorter than max_length
                    pad_length = self.max_length - chunk_input_ids.shape[0]

                    chunk_input_ids = torch.cat([
                        chunk_input_ids,
                        torch.full((pad_length,), self.tokenizer.pad_token_i
                    ])
                    chunk_attention_mask = torch.cat([
                        chunk_attention_mask,
                        torch.full((pad_length,), 0, dtype=torch.long, devic
                    ])

                self.all_chunk_encodings.append({
                    'input_ids': chunk_input_ids,
                    'attention_mask': chunk_attention_mask
                })
                self.all_chunk_labels.append(labels[original_idx])
                self.chunk_original_indices.append(original_idx)

    # Retrieve a single chunk's data and its associated original label and
    def __getitem__(self, idx):
        item = {key: val for key, val in self.all_chunk_encodings[idx].items
        item['labels'] = torch.tensor(self.all_chunk_labels[idx], dtype=torc
        item['original_index'] = torch.tensor(self.chunk_original_indices[id
        return item

    # Return the total number of chunks across all original articles
    def __len__(self):
       return len(self.all_chunk_labels)

train_dataset = NewsDataset(X_train['text'].tolist(), y_train.tolist(), robe
val_dataset = NewsDataset(X_val['text'].tolist(), y_val.tolist(), roberta_to
```

```
test_dataset = NewsDataset(X_test['text'].tolist(), y_test.tolist(), roberta
```

In [31]:

```
# Metrics tracker
class MetricsTracker:
    def __init__(self):
        self.epoch_data = []

    def log_epoch(self, epoch, metrics):
        metrics['epoch'] = epoch
        self.epoch_data.append(metrics)

    def get_dataframe(self):
        return pd.DataFrame(self.epoch_data).set_index('epoch')

metrics_tracker = MetricsTracker()

# Performance metrics calculation
def compute_metrics(p):
    preds = np.argmax(p.predictions, axis=1)
    probs = torch.softmax(torch.tensor(p.predictions), dim=1).numpy()[:, 1]

    metrics = {
        'accuracy': accuracy_score(p.label_ids, preds),
        'precision': precision_score(p.label_ids, preds, zero_division=0),
        'recall': recall_score(p.label_ids, preds, zero_division=0),
        'f1': f1_score(p.label_ids, preds, zero_division=0),
        'roc_auc': roc_auc_score(p.label_ids, probs)
   }
    return metrics
```

In [32]:

```
# Training configuration optimized for finetuning the RoBERTa transformer
training_args = TrainingArguments(
    output_dir='./roberta_results',
    run_name='roberta-fake-news-detection',
    num_train_epochs=4,
    per_device_train_batch_size=16,
    per_device_eval_batch_size=32,
    learning_rate=2e-5,
    warmup_ratio=0.06,
    weight_decay=0.01,
    eval_strategy='epoch',
    save_strategy='epoch',
    load_best_model_at_end=True,
    metric_for_best_model='f1',
    greater_is_better=True,
    logging_dir='./roberta_logs',
    logging_steps=100,
    seed=42,
    fp16=torch.cuda.is_available(),
    report_to='none',
    save_total_limit=2
)
```

In [33]:

```python
# Custom trainer for metrics tracking
class CustomTrainer(Trainer):
    def evaluate(self, *args, **kwargs):
        metrics = super().evaluate(*args, **kwargs)
        metrics_tracker.log_epoch(self.state.epoch, metrics)
        return metrics

# Initialize trainer
roberta_trainer = CustomTrainer(
    model=roberta_model,
    args=training_args,
    train_dataset=train_dataset,
    eval_dataset=val_dataset,
    compute_metrics=compute_metrics,
)

# Train model
print("Fine-tuning RoBERTa for fake news detection")
roberta_trainer.train()
```

```
Fine-tuning RoBERTa for fake news detection
```

[928/928 29:30, Epoch 4/4]

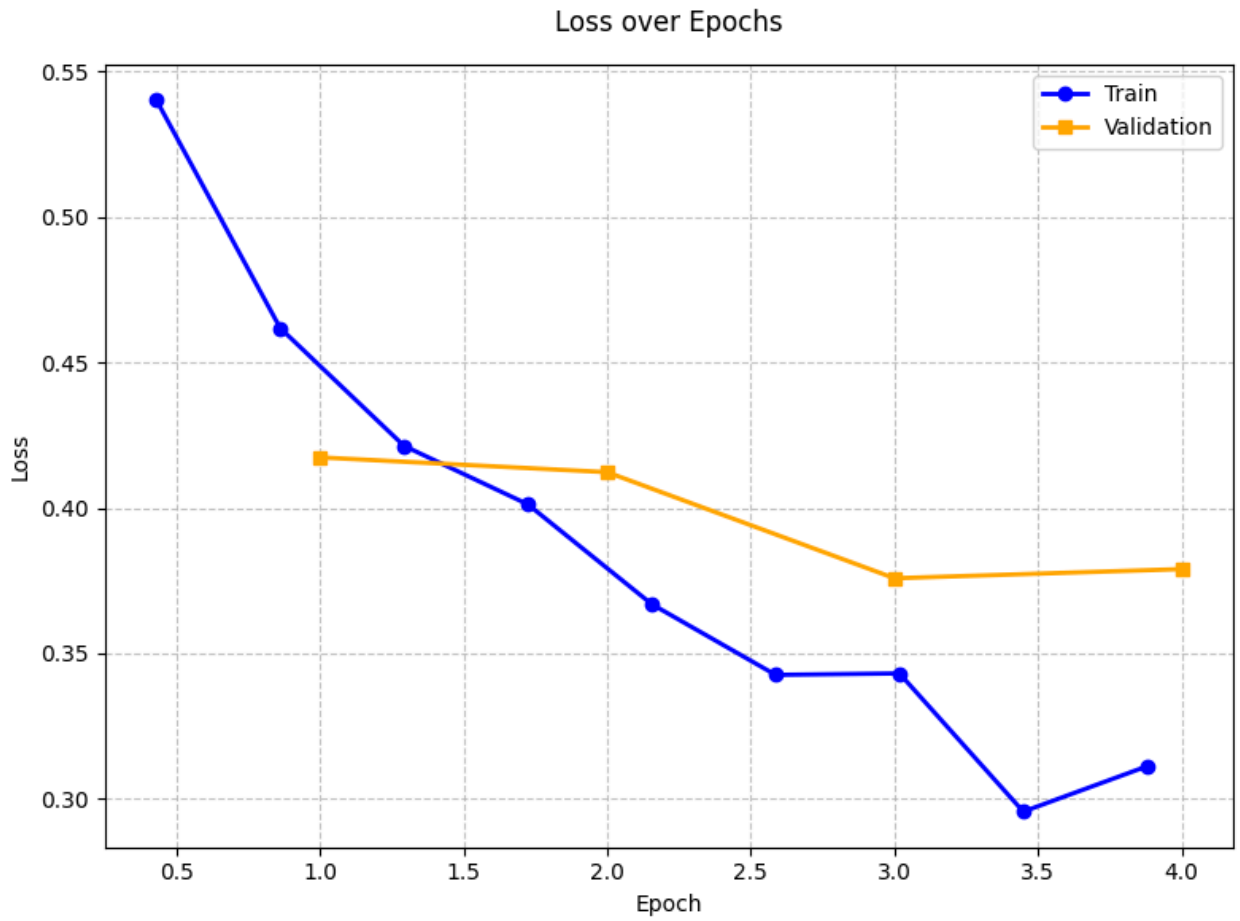
Epoch	Training Loss	Validation Loss	Accuracy	Precision	Recall	F1	Roc Auc
1	0.461800	0.417350	0.821417	0.880159	0.893274	0.886668	0.823291
2	0.401200	0.412264	0.851339	0.852189	0.979863	0.911577	0.826512
3	0.342600	0.375806	0.855748	0.870743	0.957712	0.912160	0.849978
4	0.311200	0.378955	0.859213	0.882995	0.945228	0.913052	0.853795

Out[33]:

```
TrainOutput(global_step=928, training_loss=0.3847926228210844, metrics={'train_runtime': 1773.8849, 'train_samples_per_second': 16.7, 'train_steps_per_second': 0.523, 'total_flos': 7794401903984640.0, 'train_loss': 0.3847926228210844, 'epoch': 4.0})
```

In [41]:

```
# Get training and validation metrics
train_metrics, val_metrics = extract_metrics(roberta_trainer, metrics_tracke
plt.figure(figsize=(8, 6))

if not train_metrics.empty and 'loss' in train_metrics.columns:
    plt.plot(
        train_metrics['epoch'],
        train_metrics['loss'],
        color='blue',
        label='Train',
        marker='o',
        linewidth=2)

# Plot validation loss
if 'eval_loss' in val_metrics.columns:
    plt.plot(
        val_metrics.index,
        val_metrics['eval_loss'],
        color='orange',
        label='Validation',
        marker='s',
        linewidth=2)

# Customize plot
plt.title('Loss over Epochs', fontsize=12, pad=15)
plt.xlabel('Epoch', fontsize=10)
plt.ylabel('Loss', fontsize=10)
plt.legend(fontsize=10)
plt.grid(True, linestyle='--', alpha=0.7)
plt.tick_params(axis='both', labelsize=10)
plt.tight_layout()
plt.savefig(',/images/roberta_loss_over_epochs.png', dpi=300, bbox_inches='
plt.show()
```



In [42]:

```
# Define the path to save the finetuned RoBERTa model
model_path = "/kaggle/working/roberta_model"

# Save the model
roberta_trainer.save_model(model_path)

# Save the tokenizer
roberta_tokenizer.save_pretrained(model_path)

print(f"Model saved to {model_path}")

# Save in a zipped downloadable format
!mkdir -p /kaggle/working/roberta_model_zip
!cp -r {model_path}/* /kaggle/working/roberta_model_zip/
!zip -r /kaggle/working/roberta_model.zip /kaggle/working/roberta_model_zip
```

```
Model saved to /kaggle/working/roberta_model
  adding: kaggle/working/roberta_model_zip/ (stored 0%)
  adding: kaggle/working/roberta_model_zip/tokenizer_config.json (deflated
76%)
  adding: kaggle/working/roberta_model_zip/config.json (deflated 50%)
  adding: kaggle/working/roberta_model_zip/special_tokens_map.json (deflat
ed 84%)
  adding: kaggle/working/roberta_model_zip/merges.txt (deflated 53%)
  adding: kaggle/working/roberta_model_zip/training_args.bin (deflated 5
2%)
  adding: kaggle/working/roberta_model_zip/model.safetensors (deflated 1
1%)
  adding: kaggle/working/roberta_model_zip/vocab.json (deflated 68%)
```

4. Evaluation

Objective 5: Compare models' performance on test set to select best fit model for deployment

- The respective performance of the models on the test set is compared to select the best-fit, most robust, and generalizable model for deployment.

In [65]:

```
# Function to compute metrics for a given model
def compute_model_metrics(y_true, y_pred, y_pred_proba=None, model_name=""):
    metrics = {
        'Model': model_name,
        'Accuracy': accuracy_score(y_true, y_pred),
        'Precision': precision_score(y_true, y_pred, zero_division=0),
        'Recall': recall_score(y_true, y_pred, zero_division=0),
        'F1': f1_score(y_true, y_pred, zero_division=0),
        'ROC-AUC': roc_auc_score(y_true, y_pred_proba) if y_pred_proba is no
    return metrics

# Logistic Regression
y_test_pred_lr = lr_model.predict(X_test_1)
y_test_pred_proba_lr = lr_model.predict_proba(X_test_1)[:, 1]
lr_metrics = compute_model_metrics(y_test, y_test_pred_lr, y_test_pred_proba

# XGBoost
y_test_pred_xgb = xgb_model.predict(X_test_1)
y_test_pred_proba_xgb = xgb_model.predict_proba(X_test_1)[:, 1]
xgb_metrics = compute_model_metrics(y_test, y_test_pred_xgb, y_test_pred_pro

# LSTM
y_pred_probs_lstm = lstm_model.predict(X_test_seq)
y_pred_lstm = (y_pred_probs_lstm > 0.5).astype("int32")
lstm_metrics = compute_model_metrics(y_test, y_pred_lstm, y_pred_probs_lstm

# Finetuned RoBERTa
def compute_metrics(p):
    preds = np.argmax(p.predictions, axis=1)
    probs = torch.softmax(torch.tensor(p.predictions), dim=1).numpy()[:, 1]
    return compute_model_metrics(p.label_ids, preds, probs, "Finetuned RoBER
test_results = roberta_trainer.predict(test_dataset)
roberta_metrics = compute_metrics(test_results)
y_pred = np.argmax(test_results.predictions, axis=1)
y_probs = torch.softmax(torch.tensor(test_results.predictions), dim=1).numpy

# Combine all metrics into a DataFrame
metrics_list = [lr_metrics, xgb_metrics, lstm_metrics, roberta_metrics]
results_df = pd.DataFrame(metrics_list)
results_df = results_df.round(4)
results_df = results_df[['Model', 'Accuracy', 'Precision', 'Recall', 'F1',
results_df
```

```
The history saving thread hit an unexpected error (OperationalError('attem
pt to write a readonly database')).History will not be written to the data
base.
142/142 ━━━━━━━━━━━━━━━━━━━━ 1s 7ms/step
```

Out[65]:

	Model	Accuracy	Precision	Recall	F1	ROC-AUC
0	Logistic Regression	0.8227	0.8317	0.9618	0.8920	0.8185
1	XGBoost	0.8260	0.8436	0.9470	0.8923	0.8219
2	LSTM	0.8214	0.8584	0.9166	0.8865	0.8056
3	Finetuned RoBERTa	0.8470	0.8691	0.9406	0.9035	0.8558

- The Finetuned RoBERTa transformer outperforms the other models in overall Accuracy, Precision, F1-score and ROC-AUC score justifying its superior performance in predicting whether a news article is fake or real.

In [47]:

```
# Print detailed classification reports
# Logistic Regression
print("\nClassification Report: Logistic Regression")
print(classification_report(y_test, y_test_pred_lr, target_names=['Fake (0)

# XGBoost
print("\nClassification Report: XGBoost")
print(classification_report(y_test, y_test_pred_xgb, target_names=['Fake (0

# LSTM
print("\nClassification Report: LSTM")
print(classification_report(y_test, y_pred_lstm, target_names=['Fake (0)',

# Finetuned RoBERTa
print("\nClassification Report: Finetuned RoBERTa")
print(classification_report(y_test, y_pred, target_names=['Fake (0)', 'Real
```

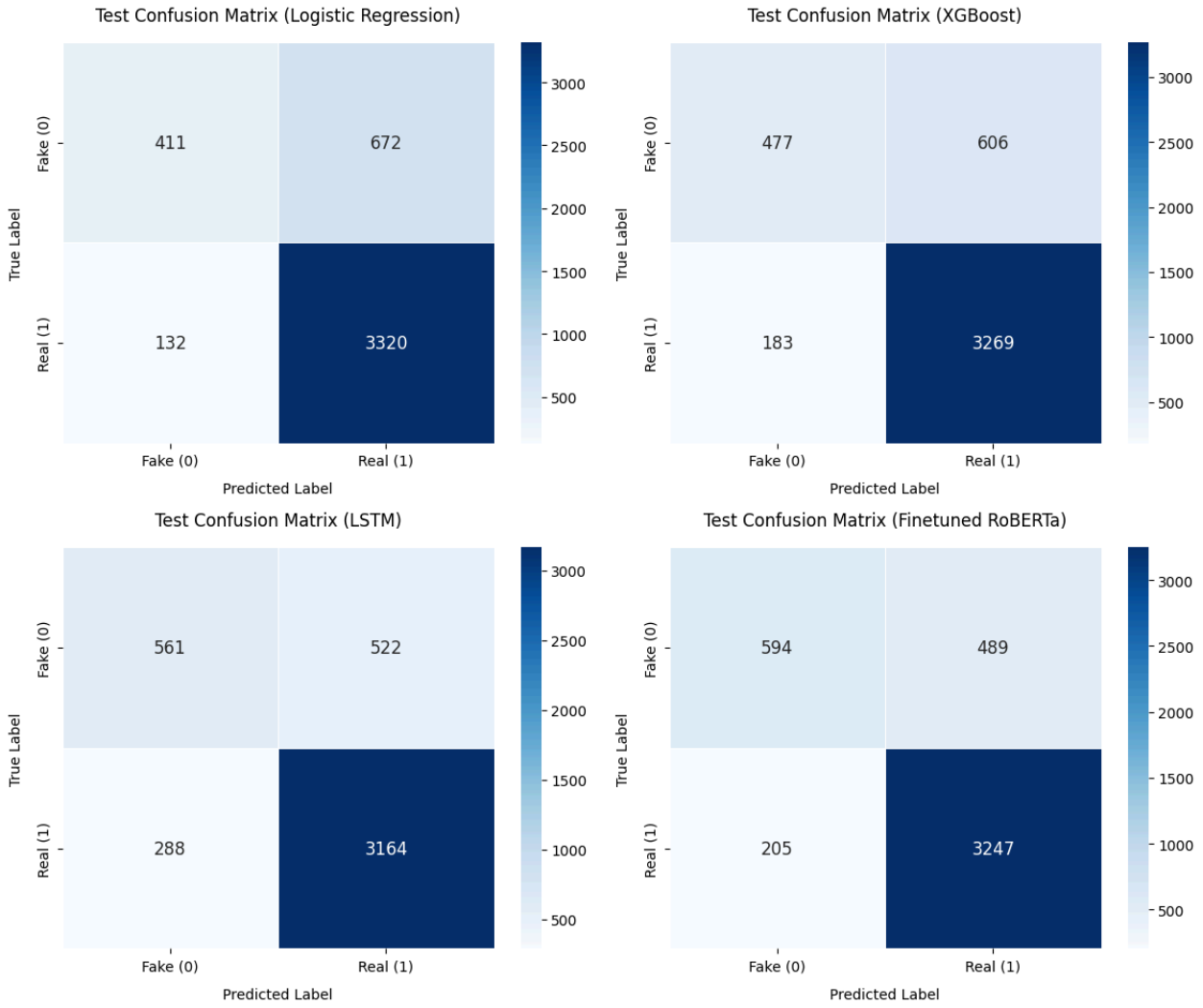
```
Classification Report: Logistic Regression
              precision    recall  f1-score   support

    Fake (0)       0.76      0.38      0.51      1083
     Real(1)       0.83      0.96      0.89      3452

    accuracy                           0.82      4535
   macro avg       0.79      0.67      0.70      4535
weighted avg       0.81      0.82      0.80      4535


Classification Report: XGBoost
              precision    recall  f1-score   support

    Fake (0)       0.72      0.44      0.55      1083
     Real(1)       0.84      0.95      0.89      3452

    accuracy                           0.83      4535
   macro avg       0.78      0.69      0.72      4535
weighted avg       0.81      0.83      0.81      4535


Classification Report: LSTM
              precision    recall  f1-score   support

    Fake (0)       0.66      0.52      0.58      1083
     Real(1)       0.86      0.92      0.89      3452

    accuracy                           0.82      4535
   macro avg       0.76      0.72      0.73      4535
weighted avg       0.81      0.82      0.81      4535


Classification Report: Finetuned RoBERTa
              precision    recall  f1-score   support

    Fake (0)       0.74      0.55      0.63      1083
     Real(1)       0.87      0.94      0.90      3452

    accuracy                           0.85      4535
   macro avg       0.81      0.74      0.77      4535
weighted avg       0.84      0.85      0.84      4535
```

- The baseline Logistic Regression model scores a macro average f1 score of 0.70 (baseline).
- The XGBoost ensemble model scores 0.72 on the macro average f1 metric.

- The LSTM model's macro average f1-score improves slightly to 0.73.
- The Finetuned RoBERTa transformer registers substantial improvement at 0.77 on the macro average f1-score highlighting its superior ability to distinguish classes.

In [48]:

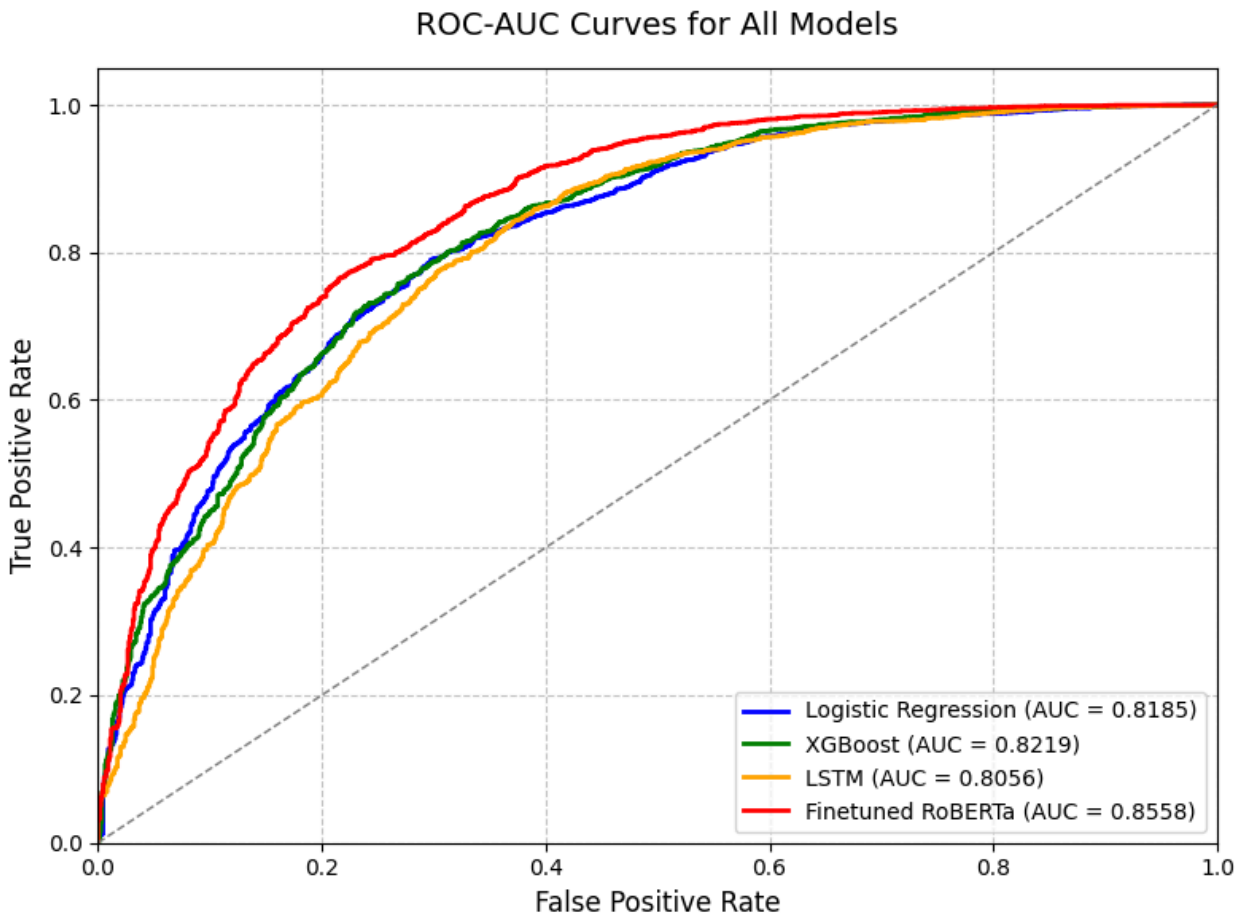
```
# Plot confusion matrices for models' predictions on test set
fig, axes = plt.subplots(2, 2, figsize=(12, 10))

# List of models, predictions, and titles
models = [
    ('Logistic Regression', y_test_pred_lr, axes[0, 0]),
    ('XGBoost', y_test_pred_xgb, axes[0, 1]),
    ('LSTM', y_pred_lstm, axes[1, 0]),
    ('Finetuned RoBERTa', np.argmax(test_results.predictions, axis=1), axes
]

# Plot confusion matrices
for model_name, y_pred, ax in models:
    # Compute confusion matrix
    cm = confusion_matrix(y_test, y_pred)

    # Plot heatmap
    sns.heatmap(
        cm,
        annot=True,
        fmt='d',
        cmap='Blues',
        xticklabels=['Fake (0)', 'Real (1)'],
        yticklabels=['Fake (0)', 'Real (1)'],
        annot_kws={'size': 12},
        linewidths=0.5,
        ax=ax
    )

    # Set titles and labels
    ax.set_title(f'Test Confusion Matrix ({model_name})', fontsize=12, pad=1
    ax.set_xlabel('Predicted Label', fontsize=10, labelpad=10)
    ax.set_ylabel('True Label', fontsize=10, labelpad=10)
    ax.tick_params(axis='both', labelsize=10)

plt.tight_layout()
plt.savefig('./images/combined-confusion-matrices-on-test-set.png', dpi=300
plt.show()
```



- The baseline model (Logistic Regression) achieves 411 true positives (TP).
- XGBoost improves to 477 TP reflecting better precision and recall.
- The LSTM model achieves 561 TP, suggesting enhanced ability to distinguish classes.
- The Finetuned RoBERTa transformer excels with 594 TP, demonstrating the highest accuracy and lowest error rates, attributable to its contextual understanding from pretraining and fine-tuning.

In [49]:

```
# Plot ROC-AUC curves
plt.figure(figsize=(8, 6))

# Define models, and their predicted probabilities.
models = [
    ('Logistic Regression', y_test_pred_proba_lr, y_test, 'blue'),
    ('XGBoost', y_test_pred_proba_xgb, y_test, 'green'),
    ('LSTM', y_pred_probs_lstm.flatten(), y_test, 'orange'),
    ('Finetuned RoBERTa', torch.softmax(torch.tensor(test_results.prediction
]

# Plot ROC curve for each model
for model_name, y_pred_proba, y_true, color in models:
    # Compute ROC curve and AUC
    fpr, tpr, _ = roc_curve(y_true, y_pred_proba)
    roc_auc = auc(fpr, tpr)

    # Plot ROC curve
    plt.plot(fpr, tpr, color=color, lw=2, label=f'{model_name} (AUC = {roc_a

# Plot random classifier line
plt.plot([0, 1], [0, 1], color='gray', linestyle='--', lw=1)

# Customize plot
plt.xlabel('False Positive Rate', fontsize=12)
plt.ylabel('True Positive Rate', fontsize=12)
plt.title('ROC-AUC Curves for All Models', fontsize=14, pad=15)
plt.legend(loc='lower right', fontsize=10)
plt.grid(True, linestyle='--', alpha=0.7)
plt.xlim([0.0, 1.0])
plt.ylim([0.0, 1.05])

# Save and show plot
plt.tight_layout()
plt.savefig('roc_auc_curves.png', dpi=300, bbox_inches='tight')
plt.show()
```



- The ROC-AUC curve for RoBERTa shows the highest area under the curve (0.856).
- It substantially outperformes Logistic Regression (0.819), XGBoost (0.822), and LSTM (0.801).

Selected Model for Deployment

The ROC-AUC curves, confusion matrices, and classification reports collectively justify selecting the Finetuned RoBERTa model for deployment. This is corroborated by the confusion matrix, which reports the highest true positives (accurate predictions for fake news). The classification report reinforces this with the finetuned transformer achieving the highest macro averaged f1-score at 0.77 showcasing balanced performance. RoBERTa's contextual understanding from pretraining and fine-tuning enhances its robustness.

5. Model Interpretability

We utilize the LIME (Local Interpretable Model-agnostic Explanations) library to interpret the predictions of a fine-tuned RoBERTa model, which classifies news articles as either class 0 (fake) or class 1 (real). We load the pre-trained RoBERTa model and its tokenizer from a specified path, setting the model to evaluation mode to disable gradient computation. A predictor function tokenizes input texts, processes them through the model, and applies a softmax function to convert logits into probability scores for the two classes.

The LIME explainer is initialized with class names 'real' and 'fake', enabling local interpretation of a selected news article (e.g., `data.iloc[99, 10]`). It generates explanations by perturbing the text and evaluating the model's predictions on these variations, using 500 samples and highlighting the top 10 features. The results are visualized as a 1x2 horizontal bar plot, with one subplot per class. Bars represent feature weights:

- Green for positive influence (supporting the class).
- Red for negative influence.

The bars are sorted by absolute weight to emphasize the most impactful words. Leveraging the LIME library provides insight into which terms drive the model's classification, enhancing interpretability of the fake/real news prediction.

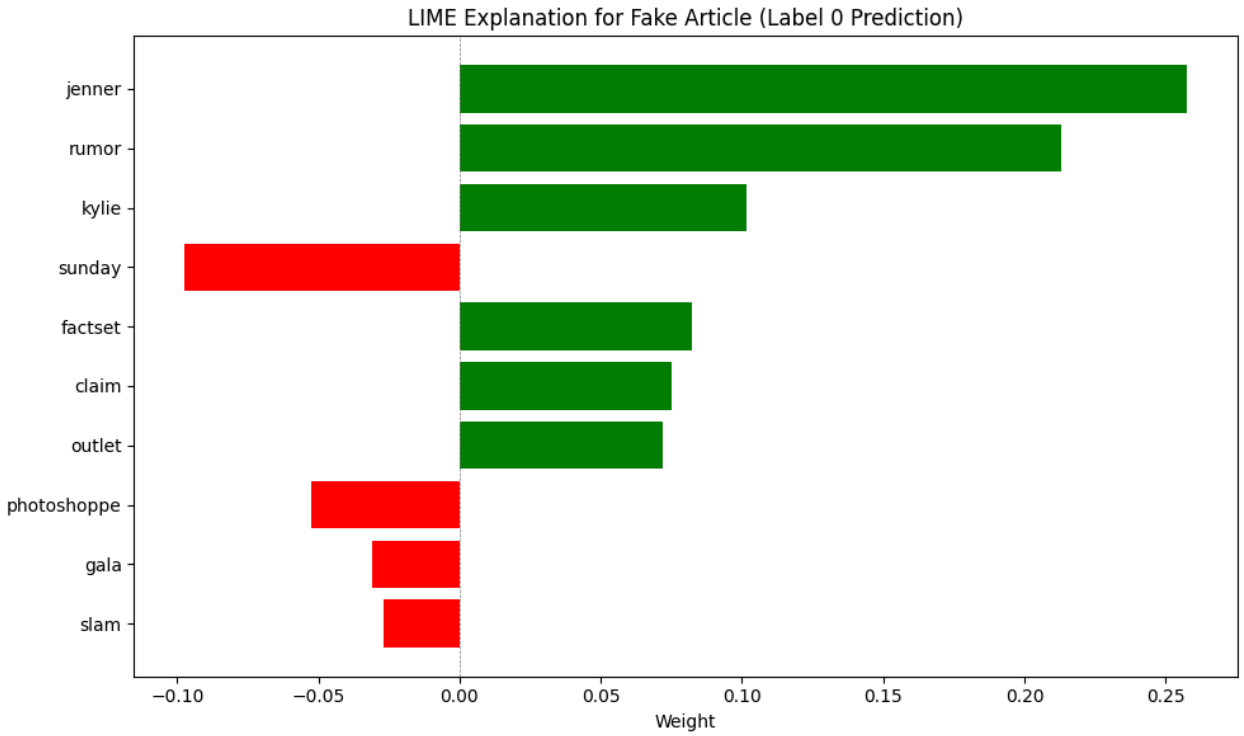
In [49]:
```
# Preview a random sample
data.iloc[20, 10]
```

Out[49]:
```
'kylie jenner slam alter photo amid pregnancy rumor material publish broad
cast rewrite redistribute fox news network llc right reserve quote display
real time delay minute market datum provide factset power implement factse
t digital solution legal statement mutual fund etf datum provide refinitiv
lipper kylie jenner metropolitan museum art costume institute gala reuters
kylie jenner slam photo publish weekend claim photoshoppe year old reality
star continue remain quiet recent pregnancy rumor jenner tweet photo sunda
y originally post daily mail say clearly alter go photoshop photo blog pap
check crooked line backgroundnd photo clearly alter jenner tweet image del
ete later retweete edit check car line daily mail publish exclusive papara
zzi photo sunday article title baby jenner kylie debut bump time hide preg
nant figure baggy clothe jenner picture black yeezy calabasa sweatshirt sw
eatpant small airport site later post dailymailcom confirm picture photosh
oppe xonline hire photographer take photo defend image jenner say real pho
toshop nofilter additive preservative khloe kardashian reportedly pregnant
addition sister kylie jenner jenner kardashian family confirm deny report
reality star expect child boyfriend travis scott news report september mon
th news outlet say old sister khloe kardashian expect child boyfriend tris
tan thompson kim kardashian confirm later month expect child surrogate sea
son trailer keep kardashian daily look news music movie television enterta
inment industry enter email click subscribe button agree fox news privacy
policy term use agree receive content promotional communication fox news u
nderstand opt time subscribe successfully subscribe newsletter material pu
blish broadcast rewrite redistribute fox news network llc right reserve qu
ote display real time delay minute market datum provide factset power impl
ement factset digital solution legal statement mutual fund etf datum provi
de refinitiv lipper'
```

In [53]:

```
from lime.lime_text import LimeTextExplainer

# Load the fine-tuned model and tokenizer
model_path = "./deployment/backend/roberta_model"
tokenizer = RobertaTokenizerFast.from_pretrained(model_path)
model = RobertaForSequenceClassification.from_pretrained(model_path)
model.eval()

# Define the LIME-compatible predictor function
def predictor(texts):
    encodings = tokenizer(texts, truncation=True, padding=True, return_tenso
    with torch.no_grad():
        outputs = model(**encodings)
    probas = torch.nn.functional.softmax(outputs.logits, dim=1).numpy()
    return probas

# LIME explainer instance
class_names = ['Fake Article', 'Real Article']
explainer = LimeTextExplainer(class_names=class_names)

# Choose an article to explain
sample_text = data.iloc[20, 10]

# Get prediction
probas = predictor([sample_text])
predicted_label = int(np.argmax(probas))  # 0 or 1
predicted_class_name = class_names[predicted_label]

# Generate explanation for the predicted label
explanation = explainer.explain_instance(
    sample_text,
    classifier_fn=predictor,
    num_features=10,
    num_samples=500,
    labels=[predicted_label])

# Extract top-10 features for the predicted label
features = explanation.as_list(label=predicted_label)
features_sorted = sorted(features, key=lambda x: abs(x[1]), reverse=True)
features, weights = zip(*features_sorted[:10])

print(f"Predicted Class: {predicted_class_name} (Label {predicted_label})")

# Plot a horizontal bar plot to visualize top-10 important features
y_pos = np.arange(len(features))
colors = ['green' if w > 0 else 'red' for w in weights]
plt.figure(figsize=(10, 6))
plt.barh(y_pos, weights, align='center', color=colors)
plt.yticks(y_pos, features)
plt.gca().invert_yaxis()
plt.xlabel('Weight')
plt.axvline(x=0, color='gray', linestyle='--', linewidth=0.5)
plt.title(f"LIME Explanation for {predicted_class_name} (Label {predicted_la
plt.tight_layout()
plt.savefig('./images/lime_explanation_barplot.png', dpi=300, bbox_inches='t
plt.show()
```

```
Predicted Class: Fake Article (Label 0)
```



6. Deployment

6.1 Thresholding Strategy

- The classification report revealed that the selected model for deployment (**Finetuned RoBERTa Transformer**) scored a precision of 0.74 and a relatively low recall of 0.55 for Fake Article predictions, based on a support of 1,083 samples.
- In comparison; the model achieved a precision of 0.87 and a recall of 0.92 for Real Article predictions based on a support of 3,452 samples.

This disparity alludes that the model is more confident in detecting real news but struggles to identify fake news. Consequently, the model's bias manifests in its high False Negatives (489 out of 1083) for class 0 predictions compared to (205 out of 3452) for class 1. Thus, we adjusted the classification thresholds for the finetuned RoBERTa transformer model to account for imbalance in the training data and improve recall for the minority class (**Fake Article (Label 0)**).

An asymmetric thresholding strategy was adopted to improve the model's ***recall*** at the expense of a slight drop in ***precision*** as follows:

- The threshold for predicting Fake Articles was lowered to **0.30**.
- The threshold for predicting Real Articles was raised to **0.75**.
- Probabilities between (**0.25 to 0.30**) for ***Label 0*** and (**0.70 to 0.75**) for ***Label 1*** were flagged for human review.

6.2 Containerization Strategy

We leveraged **FASTAPI**, **Streamlit**, and **Docker** to deploy the **fine-tuned RoBERTa Transformer Model** for fake news detection.

- **Back-end :** The **FastAPI** app (***main.py***) is responsible for model inference. It loads the saved roberta_model, incorporates the SpaCy text preprocessing pipeline, scrapes article text from a provided URL using BeautifulSoup, predicts whether it is fake or real, and generates LIME explanation results.
- **Front-end :** The **Streamlit** app (***app.py***) provides an interactive frontend web app that prompts the user to enter a news article URL for prediction. It sends the URL to the FastAPI endpoint (/predict/)

via localhost, displays the article title, prediction label, and generates a LIME explanation plot.
- **Docker :** Used to containerize the ***front-end*** and the ***back-end*** using two Dockerfiles (one for each).
  - The required Python dependencies for the front-end include: ***streamlit, requests ,*** and ***matplotlib***.
  - The required Python dependencies for the back-end include: ***fastapi, uvicorn, transformers, torch, numpy, lime, pydantic, requests, beautifulsoup4, spacy ,*** and ***lxml***.
  - The docker-compose.yml exposes ports: **"8000:8000"** to the back-end **FastAPI** (***main.py***) and ports **"8501:8501"** to the front-end **Streamlit app** (***app.py***).

7. Conclusion and Recommendations

7.1 Conclusion

This project successfully developed a robust pipeline for fake news detection. It leveraged advanced NLP techniques and multiple machine learning approaches. The dataset (15,116 news articles) was meticulously processed through feature engineering (domain extraction, text length analysis, preprocessing) and exploratory analysis. Because the target variable was imbalanced; the macro averaged f1-score was set as the success criteria metric. The Key objectives were addressed through:

- **Baseline and Ensemble Models:** Logistic Regression and XGBoost established initial performance benchmarks.
- **LSTM Network:** Captured sequential dependencies in article text.
- **RoBERTa Fine-tuning:** The standout solution achieved state-of-the-art results (***macro averaged f1-score: 0.77***), demonstrating superior contextual understanding and generalization.

The finetuned RoBERTa transformer model excels in real-world applicability owing to its effectiveness in distinguishing linguistic patterns unique to misinformation. The project underscores the viability of transformer models for NLP-driven credibility assessment, providing a scalable tool for platforms combating digital misinformation.

7.2 Recommendations

- **Baseline and Ensemble Models:** Use TF-IDF n-grams and class weighting to handle subtle linguistic cues in future iterations.
- **LSTM Network:** Address overfitting via dropout layers and bidirectional LSTMs to optimize computational efficiency.
- **RoBERTa:** Leverage pretrained embeddings to reduce training time.

8. Next Steps

1. Deploy the containerized finetuned RoBERTa model via AWS SageMaker for low-latency inference.
2. Develop browser extensions and social media plugins using the model's API for real-time credibility scoring.
3. Incorporate automation frameworks to flag low-confidence predictions for human review.